

Ultimate .NET Backend, SQL Server & OOP Interview Master Guide (100 Questions Master Edition)

1. SQL Server & Database Fundamentals (Q1–Q26)

? Q1: SQL Query — 2nd Highest Salary

```
-- Approach 1: Modern ANSI SQL (Best performance)
SELECT Salary FROM Employees
ORDER BY Salary DESC
OFFSET 1 ROWS FETCH NEXT 1 ROWS ONLY;

-- Approach 2: Window Function (DENSE_RANK)
WITH RankedSalaries AS (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) as RankNum
    FROM Employees
)
SELECT DISTINCT Salary FROM RankedSalaries WHERE RankNum = 2;
```

? Q2: SQL Query — Highest Salary per Department

```
WITH RankedSalaries AS (
    SELECT EmployeeId, DepartmentId, Salary,
           DENSE_RANK() OVER(PARTITION BY DepartmentId ORDER BY Salary DESC) as RankNum
    FROM Employees
)
SELECT EmployeeId, DepartmentId, Salary
FROM RankedSalaries
WHERE RankNum = 1;
```

? Q3: SQL Query — Find Customers Who NEVER Placed Any Orders

```
-- Approach 1: LEFT JOIN
SELECT c.CustomerId, c.Name
FROM Customers c
LEFT JOIN Orders o ON c.CustomerId = o.CustomerId
WHERE o.OrderId IS NULL;

-- Approach 2: NOT EXISTS
SELECT c.CustomerId, c.Name
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o WHERE o.CustomerId = c.CustomerId
);
```

? Q4: SQL Query — Find Duplicate Rows

```
SELECT Email, COUNT(*)
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

? Q5: SQL Query — Delete Duplicate Rows Leaving Only 1 Copy

```
WITH CTE_Duplicates AS (
    SELECT Email,
           ROW_NUMBER() OVER(PARTITION BY Email ORDER BY CustomerId) as RowNum
    FROM Customers
)
DELETE FROM CTE_Duplicates WHERE RowNum > 1;
```

? Q6: Differences Between `COUNT(*)`, `COUNT(1)`, and `COUNT(column)`

- `COUNT(*)` & `COUNT(1)` : Identical execution plans in modern SQL Server. Counts all rows including `NULL` values.
- `COUNT(column)` : Counts only rows where the specified column value is **NOT NULL**.

? Q7: Window Functions: `ROW_NUMBER()` vs `RANK()` vs `DENSE_RANK()`

- `ROW_NUMBER()` : Assigns a unique sequential integer (1, 2, 3, 4) regardless of duplicate values.
- `RANK()` : Assigns identical rank to duplicates, but **skips subsequent numbers** (1, 2, 2, 4).
- `DENSE_RANK()` : Assigns identical rank to duplicates **without skipping numbers** (1, 2, 2, 3).

? Q8: SQL Joins: `INNER`, `LEFT`, `RIGHT`, `FULL OUTER`, `CROSS`, and `SELF JOIN`

- `INNER JOIN` : Returns matching rows existing in both tables.
- `LEFT JOIN` : Returns all rows from left table + matching rows from right table (`NULL` for non-matches).
- `RIGHT JOIN` : Returns all rows from right table + matching rows from left table.
- `FULL OUTER JOIN` : Returns all rows from both tables, filling `NULL` where matches are missing.
- `CROSS JOIN` : Cartesian product producing all possible combinations.
- `SELF JOIN` : Joining a table to itself (e.g., employee to manager hierarchy).

? Q9: `WHERE` vs `HAVING` & Query Execution Order

- `WHERE` : Filters individual records **before** `GROUP BY` aggregation.
- `HAVING` : Filters aggregated groups **after** `GROUP BY`.
- **Execution Order:**

```
FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> OFFSET-FETCH
```

? Q10: UNION vs UNION ALL

- **UNION** : Combines result sets and deduplicates records (requires sorting in **tempdb** , slower).
- **UNION ALL** : Combines result sets without removing duplicates (faster, preferred in production).

? Q11: Clustered vs Non-Clustered Indexes

- **Clustered Index** : Determines physical storage order of table data on disk (only 1 per table, usually Primary Key).
- **Non-Clustered Index** : Separate logical B-Tree structure storing column values with a pointer (RID/PK) to physical data (up to 999 per table).

? Q12: Index Seek vs Index Scan & Covered Indexes

- **Index Seek** : Direct B-Tree traversal to pinpoint specific matching rows (Fastest).
- **Index Scan** : Reading through the entire index leaf pages from start to end (Slower).
- **Covered Index** : Index containing all columns requested by **SELECT** , eliminating table lookups (**Key Lookup**).

? Q13: Leftmost Prefix Rule & Non-SARGable Queries

- **Leftmost Prefix Rule** : Composite index on **(LastName, FirstName)** works for **LastName** filters, but **not** for queries filtering only on **FirstName** .
- **Non-SARGable Queries** : Wrapping indexed columns inside functions (e.g. **WHERE YEAR(CreatedDate) = 2026**) invalidates Index Seek, forcing an Index Scan.

? Q14: Why is SELECT * Bad for Performance?

- Prevents Covered Index utilization.
- Increases disk I/O, network bandwidth, and memory consumption.
- Risky if database table schema changes over time.

? Q15: ACID Principles & Transaction Isolation Levels

- **Atomicity** : All operations succeed, or all roll back (**All or Nothing**).
- **Consistency** : Database transitions validly between schema constraints and states.
- **Isolation** : Concurrent transactions execute without cross-interference.
- **Durability** : Committed data is permanently stored even during sudden power failure.
- **Isolation Levels**: **Read Uncommitted** (Dirty Reads), **Read Committed** (Default), **Repeatable Read** , **Serializable** , **Snapshot** (Row Versioning).

? Q16: DELETE VS TRUNCATE VS DROP

- **DELETE (DML)**: Deletes row-by-row, fully logged, supports **WHERE** , fires triggers, rollable.
- **TRUNCATE (DDL)**: Deallocates data pages, resets **IDENTITY** seed to 1, faster, no **WHERE** .
- **DROP (DDL)**: Permanently removes table structure and data from database.

? Q17: Stored Procedures vs User-Defined Functions (UDF)

- **Stored Procedure** : Can execute DML (**INSERT/UPDATE/DELETE**), return multiple datasets, output parameters, and manage transactions. **Faster execution** because execution plans are compiled and cached in memory.
- **UDF** : Must return a single value or table, **cannot modify database state (no DML)**, and can be invoked inside **SELECT** queries.

? Q18: Primary Key vs Unique Key Constraints

- **Primary Key** : Uniquely identifies each row, automatically creates Clustered Index, **never allows NULL** (1 per table).
- **Unique Key** : Enforces uniqueness, **allows 1 NULL value** in SQL Server (Multiple allowed per table).

? Q19: CTEs vs Temp Tables vs Table Variables

- **CTE (WITH ...)**: Temporary inline result set, lifespan limited to single execution scope.
- **Temp Table (#Temp)**: Stored in **tempdb** , supports indexes and statistics, ideal for large datasets.
- **Table Variable (@Table)**: Stored in memory/ **tempdb** , no statistics, best for small sets (<100 rows).

? Q20: Weak Entity in Database Design (ERD)

- **Weak Entity** : An entity without its own Primary Key; its identity depends on a Foreign Key to a strong parent entity (e.g. **Address** or **Dependent** belonging to an **Employee**).

? Q21: What is a View and Why Use It?

- **Concept**: A virtual table based on the result-set of an underlying **SELECT** query.
- **Benefits**: Simplifies complex queries, enhances security by restricting access to sensitive columns, and centralizes reusable query logic.

? Q22: Triggers vs Stored Procedures

- **Stored Procedure** : Must be explicitly called by name in application code or scripts.
- **Trigger** : Executes automatically as an immediate event listener on a table (**AFTER** or **INSTEAD OF**).

? Q23: What is a Cursor in SQL Server and Why Avoid It?

- **Concept:** A mechanism to iterate row-by-row over a result set.
- **Performance Impact:** Extremely slow and resource-heavy. Always prefer set-based operations and window functions.

? Q24: How Do You Monitor & Profile Live Queries?

- Use **SQL Server Profiler** or **Extended Events** to trace queries in real-time, inspect duration, logical reads, CPU time, and capture locking deadlocks.

? Q25: Relational vs NoSQL Databases

- **Relational (SQL)** : Structured tables, strict schema, joins, ACID compliant.
- **NoSQL (Document/Key-Value)** : Unstructured/JSON storage, horizontal scaling, ultra-fast simple reads/writes without complex relational joins.

? Q26: Database Normalization (1NF, 2NF, 3NF)

- **Concept:** Organizing database tables to eliminate redundancy and update anomalies.
- **Forms:** 1NF (Atomic columns + PK), 2NF (1NF + no partial key dependency), 3NF (2NF + no transitive dependency).

2. Object-Oriented Programming (OOP Core Principles & Design) (Q27–Q37)

? Q27: The 4 Core Pillars of OOP

- **Encapsulation** : Bundling data and methods into a single unit and restricting direct external access.
- **Abstraction** : Exposing essential features while hiding complex implementation details.
- **Inheritance** : Deriving new classes from existing classes for code reuse and extensibility (**IS-A**).
- **Polymorphism** : Treating derived objects as instances of their base type with specialized behavior.

? Q28: What is Encapsulation and How is it Implemented in C#?

- **Concept:** Protecting internal object state from unauthorized or invalid modification.
- **Implementation:**
 - Declare internal fields as `private`.
 - Expose controlled access via **Properties** (`get`; `set`;) with validation rules.

```
public class BankAccount
{
    private decimal _balance; // Encapsulated private state

    public decimal Balance => _balance; // Read-only access

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Must be positive");
        _balance += amount;
    }
}
```

? Q29: How Properties (`get` , `set` , `private set` , `init`) Enforce Encapsulation

- `get` / `set` : Gatekeeping reading and writing with validation logic in setters.
- `private set` : Restricting mutation strictly to internal class methods (standard for Domain Entities).
- `init` (C# 9+): Allows mutation only during object construction, enforcing immutability afterward.

? Q30: Access Modifiers in C#

- `public` : Accessible everywhere.
- `private` : Accessible only within declaring class.
- `protected` : Accessible within declaring class and derived subclasses.
- `internal` : Accessible only within same assembly/project.
- `protected internal` : Accessible within same assembly OR derived classes in other assemblies.
- `private protected` : Accessible only by derived classes within same assembly.

? Q31: Abstract Class vs Interface

- **Abstract Class** :
 - Base class that cannot be instantiated directly.
 - Can have fields, state, constructors, and implemented methods.
 - Supports single inheritance only.
- **Can an abstract class have 0 abstract methods?** Yes, completely valid.
- **Interface** :
 - Pure contract defining required behavior without holding state.
 - No constructors or instance fields.
 - Supports multiple interface implementation.
- **Can an interface contain method bodies?** Yes, since C# 8 via **Default Interface Methods** .

? Q32: Static Class VS Sealed Class VS Abstract Class

- **Static Class** : Cannot be instantiated, cannot be inherited, all members must be static.
- **Sealed Class** : Can be instantiated normally, **cannot be inherited** (closed to derivation, e.g. **string**).
- **Abstract Class** : Cannot be instantiated, **designed specifically to be inherited**.

? Q33: Polymorphism: Overloading vs Overriding

- **Method Overloading (Compile-Time)** : Same method name with different parameter signatures.
- **Method Overriding (Run-Time)** : Re-implementing a base class method using **virtual** in parent and **override** in child.

? Q34: Virtual / Override vs New (Method Overriding vs Method Hiding)

- **virtual / override** : True polymorphism. When called through a base reference, the derived method executes.
- **new (Method Hiding)**: Hides base method without polymorphism. Calling via base reference executes base method; calling via derived reference executes derived method.

? Q35: Composition over Inheritance

- **Rule**: Prefer object composition (**HAS-A**) over class inheritance (**IS-A**).
- **Reason**: Reduces tight coupling, enhances testability via mocked interfaces, and avoids fragile base class hierarchies.

? Q36: Constructors in C#

- Special initialization methods. Types: Default, Parameterized, Static (runs once per app domain), Private (used in Singletons).

? Q37: Shallow Copy vs Deep Copy

- **Shallow Copy** : Duplicates value fields; reference fields copy pointers (`MemberwiseClone`).
- **Deep Copy** : Recursively duplicates entire object graph in distinct memory locations.

3. C# Language Internals & Memory Management (Q38–Q52)

? Q38: Value Types vs Reference Types & Stack vs Heap Memory

- **Value Types** (`int`, `double`, `bool`, `char`, `struct`, `enum`): Store data directly on **Stack**. Assignment creates an independent copy.
- **Reference Types** (`class`, `interface`, `delegate`, `string`, `object`, `Array`): Store pointer on **Stack** pointing to data on **Heap**. Assignment copies reference pointer.

? Q39: `struct` vs `class` vs `record`

- **class** : Reference type (**Heap**). Default choice for domain models and services.
- **struct** : Value type (**Stack**). Small (<16 bytes), immutable structures without inheritance (`Point`, `DateTime`).
- **record** : Immutable data containers with **value-based equality**. Ideal for **DTOs**.

? Q40: String Immutability & `StringBuilder`

- **Immutability**: Strings cannot be changed after creation for security, thread safety, and string interning (`String Interning`).
- **Loop Concatenation Pitfall**: Using `+` in loops allocates a new object on Heap every iteration, creating heavy GC pressure.
- **Solution**: Use `StringBuilder` (mutable `char[]` buffer).

? Q41: Boxing & Unboxing

- **Boxing** : Converting value type to reference type (`object obj = 10;`). Allocates on Heap.
- **Unboxing** : Extracting value type from reference type (`int x = (int)obj;`).
- **Solution**: Use generic collections (`List`) instead of non-generic collections (`ArrayList`).

? Q42: Garbage Collection (GC) Generations & Heaps

- **Gen 0** : Short-lived objects (local variables).
- **Gen 1** : Buffer generation between short-lived and long-lived objects.
- **Gen 2** : Long-lived objects (static objects, singletons).
- **LOH (Large Object Heap)** : Objects >85,000 bytes, collected during Gen 2 cycles.
- **POH (Pinned Object Heap) (.NET 5+)**: Objects pinned in memory to prevent GC movement for native interop.

? Q43: Memory Leaks in C# Applications

- Unsubscribed event handlers (`Events`).
- Static references holding onto objects forever.
- Unclosed unmanaged resources (file handles, streams).
- `Captive Dependency` in DI containers.

? Q44: `IDisposable` & `using` Statement

- `IDisposable` declares `Dispose()` . The `using` statement compiles into a `try-finally` block guaranteeing `Dispose()` execution.

? Q45: `ref` , `out` , and `in` Parameters

- `ref` : Pass-by-reference; must be initialized before passing.
- `out` : Pass-by-reference; must be assigned inside the method (`TryParse`).
- `in` : Pass-by-reference for readonly access; avoids copying large structs.

? Q46: `Span` and `ReadOnlySpan`

- Contiguous memory slicing directly on the Stack with **zero Heap allocations**.

? Q47: `throw` vs `throw ex`

- `throw` : Preserves original exception stack trace (Correct).
- `throw ex` : Resets stack trace to current line, hiding the original failure point (Antipattern).

? Q48: `const` vs `readonly`

- `const` : Evaluated at **compile-time**. Value baked into calling assembly. Must be primitive/string.
- `readonly` : Evaluated at **runtime** (in constructor). Supports reference types and dynamic assignment.

? Q49: `var` vs `dynamic` vs `object`

- `var` : Statically typed at compile-time with full IntelliSense.
- `object` : Base reference type requiring explicit casting.
- `dynamic` : Resolved at runtime via DLR; bypasses compile-time checks.

? Q50: Thread vs Task and .Result / .Wait() Starvation

- **Thread** : Heavy OS-level thread (~1MB stack).
- **Task** : High-level abstraction executing on **ThreadPool** . Awaited I/O operations release threads back to pool.
- **.Result / .Wait() Hazard**: Synchronously blocks pool threads, inducing **Thread-Pool Starvation** under high load.

? Q51: Why async void is Dangerous

- Cannot be awaited (**Fire-and-Forget**).
- Unhandled exceptions bypass caller task handlers and crash the process.
- **Only valid use case**: UI / Framework Event Handlers.

? Q52: .NET Framework vs .NET Core / .NET 8+

- Cross-platform, high-performance Kestrel server, built-in DI & Middleware pipeline, open-source.

4. SOLID Principles & Design Patterns (Q53–Q62)

? Q53: Single Responsibility Principle (SRP)

- A class should have only one reason to change (Separate **OrderService** from **EmailService** and **OrderRepository**).

? Q54: Open/Closed Principle (OCP)

- Open for extension, closed for modification (Use Strategy Pattern with **IPaymentProcessor**).

? Q55: Liskov Substitution Principle (LSP)

- Derived classes must be completely substitutable for their base class without altering program correctness.

? Q56: Interface Segregation Principle (ISP)

- Split monolithic interfaces into fine-grained client-specific contracts (**IReadRepository** vs **IWriteRepository**).

? Q57: Dependency Inversion Principle (DIP)

- High-level modules must depend on abstractions (**Interfaces**), not on concrete implementations.

? Q58: Design Pattern Categories

- **Creational:** Object instantiation ([Singleton](#), [Factory Method](#), [Abstract Factory](#), [Builder](#)).
- **Structural:** Class/Object composition ([Adapter](#), [Decorator](#), [Proxy](#), [Repository](#), [Unit of Work](#)).
- **Behavioral:** Object communication and responsibilities ([Strategy](#), [Observer](#), [Mediator](#)).

? Q59: Thread-Safe Singleton Implementation

- Make constructor [private](#) , expose public static instance property, and use [Lazy](#) for thread-safe deferred initialization.

? Q60: The Repository Pattern Definition

- **Interview Definition:** "A layer that separates data access logic from business logic."
- **Why use it over DbContext:** Decouples application from EF Core, centralizes queries, and allows clean mocking during Unit Testing.

? Q61: Unit of Work Pattern

- Coordinates multiple repositories across a single transaction to ensure atomic commits ([All or Nothing](#)).

? Q62: Mediator Pattern & MediatR

- Decouples sender and receiver by dispatching requests to dedicated handlers via an in-process mediator.

5. Modern Architecture (Clean Architecture, DDD, CQRS, Microservices) (Q63–Q71)

? Q63: Clean / Onion Architecture Layers

- **Domain:** Core business entities and domain rules (Independent of everything).
- **Application:** Use cases, DTOs, interfaces (Orchestration).
- **Infrastructure:** External implementations (EF Core, DbContext, APIs, Caching).
- **Presentation:** API Controllers / UI.
- **Rule:** All dependencies point **inward toward Domain**.

? Q64: Use Cases vs Business Logic

- [Business Logic](#) (**Domain**): Invariant rules (e.g. "Balance cannot fall below zero").
- [Use Cases](#) (**Application**): Workflow orchestration (e.g. "Create Order -> Validate -> Charge Card -> Save DB -> Send Email").

? Q65: Domain-Driven Design (DDD) Core Concepts

- **Ubiquitous Language** : Standardized shared vocabulary between business experts and developers.
- **Bounded Context** : Explicit boundary enclosing a specific domain model.
- **Entity** : Object defined by unique identity (**Id**).
- **Value Object** : Immutable object defined by its attributes without identity.
- **Aggregate Root** : Primary entity governing modifications and consistency inside an aggregate cluster.
- **Rich Domain Model** : Domain classes containing behavior and validation methods rather than plain property getters/setters.

? Q66: CQRS Pattern

- Separates **Commands** (Write operations) from **Queries** (Read operations) to optimize scaling and performance independently.

? Q67: When NOT to Use Microservices & The Modular Monolith

- **Avoid Microservices**: For small teams, simple domains, or low DevOps maturity due to distributed network overhead and consistency complexity.
- **Modular Monolith** : Single unified deployable organized internally into strict bounded modules (Best starting architecture).

? Q68: Database per Service Pattern & Inter-Service Communication

- Each service encapsulates its own private database (No cross-service SQL joins).
- **Synchronous**: REST / gRPC.
- **Asynchronous**: Message Brokers (**RabbitMQ**, **Kafka**, **Azure Service Bus**).

? Q69: Circuit Breaker Pattern & Polly

- Automatically trips open to halt calls to a failing downstream dependency, allowing it time to recover. Implemented via **Polly** in .NET.

? Q70: Result Pattern vs Exceptions

- Return **Result** for expected business validation failures to avoid costly exception stack trace overhead.

? Q71: Engineering Approach When Building an App From Scratch

- Understand business requirements -> Choose DB & draw ERD -> Pick Architecture (**Clean/DDD**) -> Implement Domain -> Implement Application -> Expose API.

6. ASP.NET Core Web API & Security (Q72–Q85)

? Q72: Request Pipeline & Middleware Order

```
Kestrel -> Exception Handling -> HTTPS -> Static Files -> Routing -> CORS -> Authentication -> Authorization -> Action Filters -> Controller -> Response
```

? Q73: DI Lifetimes & Captive Dependencies

- **Transient** : New instance every resolution.
- **Scoped** : One instance per HTTP request.
- **Singleton** : One instance for application lifetime.
- **Captive Dependency** : Injecting **Scoped** into **Singleton** traps the scoped instance in memory forever.

? Q74: Authentication vs Authorization

- **Authentication** : Verifies identity (Who are you?) -> `AddAuthentication()` .
- **Authorization** : Verifies permissions (What can you do?) -> `AddAuthorization()` , `[Authorize]` .

? Q75: Claims & Refresh Tokens in APIs

- **Claims** : Key-value attributes embedded in JWT asserting user identity and roles.
- **Refresh Token** : Long-lived secure token stored server-side to reissue short-lived JWTs without requiring re-login.

? Q76: GET vs POST vs PUT vs PATCH (Data Transport Nuance)

- GET carries data in URL query string; POST/PUT/PATCH carry payloads in request body. PUT replaces full resource; PATCH applies partial modifications via JSON Patch.

? Q77: Never Put Sensitive Data in URL Query Strings

- Logged in plain text in server logs, proxy logs, and browser history.

? Q78: HTTP Status Code Ranges

- **2xx** : Success (**200 OK** , **201 Created** , **204 No Content**).
- **4xx** : Client Error (**400 Bad Request** , **401 Unauthorized** , **403 Forbidden** , **404 Not Found**).
- **5xx** : Server Error (**500 Internal Server Error**).

? Q79: Async Background Job Status Codes (201 vs 202)

- **201 Created** : When resource creation completes synchronously.
- **202 Accepted** : When long-running job is queued for asynchronous execution.

? Q80: In-Memory vs Distributed Caching (Redis)

- In-Memory is local and lost on restart. Redis is centralized, resilient to app restarts, and shared across load-balanced replicas.

? Q81: Cache Stampede (Thundering Herd) & HybridCache

- Fix: In-flight query deduplication or .NET 9 **HybridCache** to ensure only one thread recalculates a cold cache key.

? Q82: Idempotent Payment Endpoints

- Client sends unique **Idempotency Key** (GUID). Server stores key in transaction, returning cached result upon retries.

? Q83: Structured Logging with Serilog

- Log structured JSON payloads to specialized sinks (**Seq**, **Elasticsearch**) rather than raw SQL tables.

? Q84: SignalR Real-Time Communication

- Provides bi-directional WebSockets communication between server and connected clients via Hubs.

? Q85: ViewBag vs ViewData vs TempData

- ViewBag (dynamic) & ViewData (dictionary) last 1 request. TempData persists across redirects via Session.

7. Entity Framework Core & LINQ (Q86–Q94)

? Q86: ADO.NET vs EF Core vs Dapper

- **ADO.NET** : Raw SQL, manual connection/mapping, fastest performance, highest development overhead.
- **EF Core** : Full ORM, change tracking, migrations, LINQ provider (Industry standard).
- **Dapper** : Micro-ORM, high-speed object mapping without change tracking.

? Q87: Database-First in EF Core (Scaffolding)

- Generate entity classes and DbContext from existing database via **Scaffold-DbContext** .

? Q88: Rolling Back Applied EF Migrations

- Run `Update-Database` followed by `Remove-Migration`.

? Q89: `IEnumerable` vs `IQueryable` (Scenario Version)

- `IQueryable` executes SQL filter on database server. `IEnumerable` pulls rows into memory first.

? Q90: `FirstOrDefault()` vs `SingleOrDefault()`

- `FirstOrDefault()` returns first match. `SingleOrDefault()` throws if more than 1 match is found.

? Q91: Deferred Execution in LINQ

- Execution only occurs upon enumeration (`ToList`, `Count`, `FirstOrDefault`, `foreach`).

? Q92: `AsNoTracking()` Performance Advantage

- Disables Change Tracker snapshots for read-only queries, reducing memory usage and CPU execution time by over 40%.

? Q93: Loading Strategies (Eager vs Lazy vs Explicit)

- Eager (`.Include()`) prevents N+1 query problems. Lazy delays loading until property access.

? Q94: JSON Columns in EF Core

- Store complex JSON documents inside single SQL column mapped via `.ToJson()`.

8. Performance Troubleshooting, Testing & Behavioral (Q95–Q100)

? Q95: Scenario: Slow Loading Page Troubleshooting

- Check Network Tab payload size -> Check LINQ projection (`.Select()`) -> Apply `AsNoTracking()` -> Implement DB Pagination (`Skip/Take`) -> Inspect DB Indexes with SQL Profiler.

? Q96: xUnit: `[Fact]` vs `[Theory]`

- `[Fact]` for single tests; `[Theory]` with `[InlineData]` for data-driven parametrized tests (`Arrange-Act-Assert`).

? Q97: Mocking with Moq

- Mock dependencies to isolate unit tests: `mockRepo.Setup(r => r.GetByIdAsync(1)).ReturnsAsync(...)`.

? Q98: Unit Tests vs Integration Tests

- Unit Tests test isolated units with mocks. Integration Tests test real DB and middleware via `WebApplicationFactory` & `Testcontainers`.

? Q99: The 90-Second "Tell Me About Yourself" Pitch

- Concise professional narrative highlighting your transition to Backend .NET, strong technical foundation, and real-world projects (`ECommerce`, `GymManagement`, `LibrarySystem`).

? Q100: How to Handle Unknown Questions Professionally

- Never guess a wrong answer. State: "I don't recall the exact implementation syntax right now, but the underlying core concept works by [state concept if known], and I can look up the API docs immediately."